Chapter 13: Component Communication

1. Overview

Angular applications are trees of components, and almost nothing useful happens without components talking to each other. A parent needs to hand data down to a child. A child needs to tell a parent that something happened. Two siblings that don't even know about each other need to stay in sync. A deeply nested grandchild needs to reach a service that lives far outside its own view.

Angular gives you a small, closed set of tools to solve every one of these problems:

- `@Input()` / `input()` — parent pushes data down to a child.
- `@Output()` / `output()` **with** `EventEmitter` — child pushes events up to a parent.
- `@ViewChild()` / `@ContentChild()` (and their `*Children` plural forms, plus the signal-based `viewChild()` / `contentChild()`) — a parent reaches directly into a child's public API.
- **Template reference variables (`#ref`)** — a parent template reaches into a child *declaratively*, without any component-class code.
- **Shared service with a `Subject` / `BehaviorSubject` /signal** — the universal solution for siblings, unrelated components, and any communication that doesn't fit a strict parent-child shape.

The exam-question version of this chapter's thesis: **Input/Output work only between direct parent and child. Everything else — siblings, grandparent-to-grandchild, unrelated components, cross-cutting state — should go through a shared service.** ViewChild/ContentChild and template variables are the exception that lets a parent call methods on a child directly, bypassing the Input/Output data-flow contract.

This chapter covers both the classic decorator-based API (`@Input`, `@Output`, `@ViewChild`) that has existed since Angular 2, and the modern **signal-based APIs** (`input()`, `output()`, `viewChild()`, `model()`) introduced in Angular 17.1+ and stabilized in Angular 19, which are now the recommended default for new code.

---

2. Core Concepts

2.1 Parent-to-Child: `@Input()`

The decorator form binds a class property so it can be set from a parent's template binding.

```
@Input() name!: string;
@Input({ required: true }) userId!: string; // Angular 16+: compile-time required check
@Input() set count(value: number) { this._count = value * 2; } // setter intercepts every write
```

Key rules:

- Only **one-way** data flow: parent → child. The child must never mutate an `@Input` and expect the parent to see it reflected automatically (primitives won't propagate back at all; mutating an object input will affect the parent's object too, silently, because it's the same reference — this is a gotcha, not a feature).
- `@Input()` can specify a public alias different from the internal property name: `@Input('userName') name: string;`

- `required: true` (Angular 16+) makes Angular's compiler emit an error if the parent doesn't bind it.
- `transform` (Angular 16+) lets you coerce incoming values, e.g. `@Input({ transform: booleanAttribute }) disabled = false;`

2.2 Parent-to-Child: `input()` signal function (Angular 17.1+)

```
name = input<string>();              // optional, type | undefined
name = input.required<string>();     // required, compile + runtime enforced
count = input(0, { transform: (v: string) => Number(v) });
```

`input()` returns a `Signal<T>`, read as `this.name()`. Differences from the decorator:

- It's **read-only** by construction — you cannot "set" a signal input from inside the child (no more accidental two-way mutation footguns from setters).
- Because it's a signal, it composes naturally with `computed()` and `effect()`, and it participates in Angular's fine-grained reactivity (no `ngOnChanges` boilerplate needed just to react to a change — use `effect()` or `computed()` instead).
- `input.required()` is enforced at compile time via the Angular language service/compiler when it can statically see the usage, and always at runtime.

2.3 Child-to-Parent: `@Output()` + `EventEmitter`

```
@Output() itemSelected = new EventEmitter<Item>();

selectItem(item: Item) {
  this.itemSelected.emit(item);
}
```

Parent template:

```
<app-item-list (itemSelected)="onItemSelected($event)"></app-item-list>
```

`EventEmitter<T>` extends RxJS `Subject<T>` (technically `EventEmitter` wraps/extends `Subject` behavior with sync/async emission). It is **not** meant for general-purpose reactive streams inside a component's own logic — it exists specifically to be the type behind `@Output()`.

2.4 Child-to-Parent: `output()` function (Angular 17.3+)

```
itemSelected = output<Item>();
// emit exactly the same way
this.itemSelected.emit(item);
```

`output()` is **not** a signal — it returns an `OutputEmitterRef<T>`, a lighter-weight emitter that is not built on RxJS `Subject`. It's used the same way in the template (`(itemSelected)="..."`). Advantages over `EventEmitter`:

- No accidental RxJS operator usage misleading people into treating it like a stream (`.pipe()`,

`.subscribe()` with manual unsubscribe footguns).

- Automatically completes/cleans up when the component is destroyed — no manual `ngOnDestroy` unsubscribe needed for consumers that use the `(event)` template syntax (Angular manages that binding's lifecycle already); and internally it avoids leaking subscriptions the way a raw `Subject` can if someone calls `.subscribe()` manually and forgets to tear it down.
- You can convert it to an Observable when needed via `outputToObservable()` from `@angular/core/rxjs-interop`.

2.5 Two-Way Binding: `@Input()` + `@Output()` pair, and `model()`

Classic banana-in-a-box `[(ngModel)]`-style two-way binding is just sugar for an Input/Output pair following the `x` / `xChange` naming convention:

```
@Input() value!: number;
@Output() valueChange = new EventEmitter<number>();
```

```
<app-counter [(value)]="total"></app-counter>
<!-- desugars to -->
<app-counter [value]="total" (valueChange)="total = $event"></app-counter>
```

Angular 17.2+ offers `model()` as first-class support for this pattern:

```
value = model<number>(0);
value = model.required<number>();

increment() {
  this.value.update(v => v + 1); // updates local signal AND emits valueChange automatically
}
```

`model()` is a **writable** signal (unlike `input()`), and writing to it both updates the child's local value and emits the change event to the parent in one step — it is Angular's built-in two-way-binding primitive and is the direct signal-based replacement for the Input+Output-pair convention.

2.6 Parent Reaching Into Child: `@ViewChild()` / `@ViewChildren()`

Use when the parent needs to call a method or read a property on a child component instance directly — something Input/Output data flow can't express (e.g., "reset this child form now," "focus this child input," "read the child's computed total on demand").

```
@ViewChild(ChildComponent) child!: ChildComponent;
@ViewChild('refName') childByRef!: ElementRef;
@ViewChild(ChildComponent, { static: true }) childStatic!: ChildComponent; // available in ngOnInit
@ViewChildren(ItemComponent) items!: QueryList<ItemComponent>;
```

Availability timing: dynamic queries (`static: false`, the default) are populated only after `ngAfterViewInit`; reading `this.child` in `ngOnInit` gives `undefined`. `static: true` is only valid when the queried element is NOT inside an `*ngIf` / `*ngFor` — it must always be present in the template.

Signal-based equivalent (Angular 17.3+ `viewChild()`):

```
child = viewChild(ChildComponent);              // Signal<ChildComponent | undefined>
child = viewChild.required(ChildComponent);     // Signal<ChildComponent>, throws if absent
items = viewChildren(ItemComponent);            // Signal<readonly ItemComponent[]>
```

These are readable immediately as signals but resolve to a real value only once the view has been rendered; reading `child()` before that returns `undefined` for the non-required variant, same underlying timing constraint, just expressed as a signal instead of a lifecycle-hook-guarded field.

2.7 Content Projection Communication: `@ContentChild()` / `@ContentChildren()`

Same idea as ViewChild, but for elements/components projected into a component via `<ng-content>` (i.e., authored by the *consumer* of your component, not by your component's own template).

```
@ContentChild(TabComponent) firstTab!: TabComponent;
@ContentChildren(TabComponent) tabs!: QueryList<TabComponent>;
```

Timing: available in `ngAfterContentInit`, not `ngAfterViewInit`. This is one of the most commonly confused points in interviews — **content** queries resolve in the **content** hooks, **view** queries resolve in the **view** hooks.

Signal form: `contentChild()`, `contentChildren()` (Angular 17.3+), same semantics as `viewChild` / `viewChildren` but querying projected content.

2.8 Template Reference Variable Communication

A `#ref` on a child component/element, used directly in the parent's own template — no component TypeScript code required at all for the read:

```
<app-counter #counterRef></app-counter>
<button (click)="counterRef.increment()">Increment</button>
<p>{{ counterRef.count }}</p>
```

This works because the parent template is compiled with knowledge of the child component instance the reference points to; it exposes the full public surface of that component instance. It's declarative, lightweight, and perfect for simple "click this button, call that method on the sibling element in the same template" cases — but it **cannot** be used outside the template (no equivalent access from the parent's TS class without also declaring a `@ViewChild`), and it doesn't work across component boundaries (you can't reference a variable from a *different* template).

2.9 Service-Based Communication (Siblings / Unrelated Components / Cross-Cutting State)

This is the general-purpose answer whenever components are **not** in a direct parent-child relationship, or when many-to-many communication is needed. A shared, injectable service (typically provided at a common ancestor or in root) exposes state via a `Subject` / `BehaviorSubject` / `signal`:

```
@Injectable({ providedIn: 'root' })
export class CartService {
  private itemsSubject = new BehaviorSubject<Item[]>([]);
  items$ = this.itemsSubject.asObservable();

  // Signal-based alternative:
  items = signal<Item[]>([]);

  addItem(item: Item) {
    this.itemsSubject.next([...this.itemsSubject.value, item]);
    this.items.update(list => [...list, item]);
  }
}
```

Any component (sibling, cousin, completely unrelated part of the tree) injects `CartService` and either subscribes to `items$` or reads the `items` signal. This decouples the components entirely — neither knows the other exists; both only know about the service contract.

Choice of subject type matters:

- `Subject` — no initial value, only emits to subscribers active at emission time (good for one-off events/commands, like "close all modals now").
- `BehaviorSubject` — has a current value, late subscribers get the last emitted value immediately (good for state that has a sensible "current" reading, like "current logged-in user").
- `ReplaySubject` — replays the last N emissions to new subscribers (good when you need history, not just current state).
- `signal` — the modern default for local/shared state in Angular 16+; no subscription management, integrates with `computed` / `effect`, read synchronously, no manual unsubscribe ever needed.

2.10 Decision Matrix: Which Pattern For Which Relationship

Relationship	Preferred pattern
Parent → direct child, simple data	`@Input()` / `input()`
Direct child → parent, simple event	`@Output()` / `output()`
Two-way sync between parent and direct child	`model()` (or Input+Output pair)
Parent needs to *command* a child (call a method, force a reset)	`@ViewChild()` / `viewChild()` or template ref variable
Parent needs to read/react to projected content	`@ContentChild()` / `contentChild()`
Same-template imperative call, no class code	Template reference variable
Siblings	Shared service
Unrelated components anywhere in the tree	Shared service

Relationship	Preferred pattern
Grandparent ↔ grandchild (skipping levels)	Shared service (avoid "input drilling" through every intermediate level)
App-wide state (auth, theme, cart)	Shared service (`providedIn: 'root'`) with `signal` / `BehaviorSubject`

3. Code Examples

3.1 `@Input()` / `@Output()` — classic parent-child pair

```
// child.component.ts
import { Component, EventEmitter, Input, Output } from '@angular/core';

@Component({
  selector: 'app-rating',
  standalone: true,
  template: `
    <div>
      @for (star of stars; track $index) {
        <span (click)="rate($index + 1)">{{ $index < value ? '★' : '☆' }}</span>
      }
    </div>
  `,
})
export class RatingComponent {
  @Input() value = 0;
  @Output() valueChange = new EventEmitter<number>();
  stars = [1, 2, 3, 4, 5];

  rate(newValue: number) {
    this.value = newValue;
    this.valueChange.emit(newValue); // Output + "xChange" naming enables [(value)] sugar
  }
}
```

```
<!-- parent.component.html -->
<app-rating [(value)]="productRating"></app-rating>
<p>You rated: {{ productRating }}</p>
```

3.2 Signal-based `input()` / `output()` equivalent

```
import { Component, input, output } from '@angular/core';

@Component({
  selector: 'app-rating',
  standalone: true,
  template: `
```

```
    <div>
      @for (star of stars; track $index) {
        <span (click)="rate($index + 1)">{{ $index < value() ? '★' : '☆' }}</span>
      }
    </div>
  `,
})
export class RatingComponent {
  value = input(0);
  valueChange = output<number>();
  stars = [1, 2, 3, 4, 5];

  rate(newValue: number) {
    this.valueChange.emit(newValue);
  }
}
```

3.3 `model()` — cleanest two-way binding

```
import { Component, model } from '@angular/core';

@Component({
  selector: 'app-rating',
  standalone: true,
  template: `
    @for (star of stars; track $index) {
      <span (click)="value.set($index + 1)">{{ $index < value() ? '★' : '☆' }}</span>
    }
  `,
})
export class RatingComponent {
  value = model(0); // no separate Output needed at all
  stars = [1, 2, 3, 4, 5];
}
```

```
<app-rating [(value)]="productRating"></app-rating>
```

3.4 `@ViewChild()` — parent commanding a child

```
@Component({
  selector: 'app-search-box',
  standalone: true,
  template: `<input #inputEl [value]="text" (input)="text = inputEl.value" />`,
})
export class SearchBoxComponent {
  @ViewChild('inputEl') inputEl!: ElementRef<HTMLInputElement>;
  text = '';
  clear() {
    this.text = '';
    this.inputEl.nativeElement.focus();
```

```
  }
}

@Component({
  selector: 'app-toolbar',
  standalone: true,
  imports: [SearchBoxComponent],
  template: `
    <app-search-box #search></app-search-box>
    <button (click)="search.clear()">Clear</button>
  `,
})
export class ToolbarComponent {
  @ViewChild(SearchBoxComponent) search!: SearchBoxComponent;

  ngAfterViewInit() {
    console.log(this.search.text); // safe here; undefined in ngOnInit
  }
}
```

Note the toolbar template above actually demonstrates *both* the template reference variable (`#search` used directly in `(click)="search.clear()"`) and the `@ViewChild` decorator (used for TS-side access in `ngAfterViewInit`) — showing they can coexist and solve slightly different needs on the same element.

3.5 `@ContentChild()` — reading projected content

```
@Component({
  selector: 'app-tab',
  standalone: true,
  template: `<div [hidden]="!active"><ng-content></ng-content></div>`,
})
export class TabComponent {
  @Input() title = '';
  active = false;
}

@Component({
  selector: 'app-tab-group',
  standalone: true,
  template: `
    <div class="tab-headers">
      @for (tab of tabs; track tab.title) {
        <button (click)="select(tab)">{{ tab.title }}</button>
      }
    </div>
    <ng-content></ng-content>
  `,
})
export class TabGroupComponent implements AfterContentInit {
  @ContentChildren(TabComponent) tabQueryList!: QueryList<TabComponent>;
  tabs: TabComponent[] = [];
```

```
  ngAfterContentInit() {
    this.tabs = this.tabQueryList.toArray();
    if (this.tabs.length) this.tabs[0].active = true;
  }

  select(tab: TabComponent) {
    this.tabs.forEach(t => (t.active = t === tab));
  }
}
```

3.6 Service-based communication for siblings

```
@Injectable({ providedIn: 'root' })
export class NotificationBusService {
  private messages = new Subject<string>();
  messages$ = this.messages.asObservable();
  notify(msg: string) { this.messages.next(msg); }
}

@Component({ selector: 'app-publisher', standalone: true, template: `<button
(click)="send()">Send</button>` })
export class PublisherComponent {
  constructor(private bus: NotificationBusService) {}
  send() { this.bus.notify('Hello from publisher'); }
}

@Component({ selector: 'app-subscriber', standalone: true, template: `<p>{{ lastMessage }}
</p>` })
export class SubscriberComponent implements OnInit, OnDestroy {
  lastMessage = '';
  private sub!: Subscription;
  constructor(private bus: NotificationBusService) {}

  ngOnInit() {
    this.sub = this.bus.messages$.subscribe(msg => (this.lastMessage = msg));
  }
  ngOnDestroy() {
    this.sub.unsubscribe(); // mandatory — services outlive components
  }
}
```

Signal-based sibling communication (no manual unsubscribe at all):

```
@Injectable({ providedIn: 'root' })
export class NotificationBusService {
  lastMessage = signal('');
  notify(msg: string) { this.lastMessage.set(msg); }
}

@Component({ selector: 'app-subscriber', standalone: true, template: `<p>{{
bus.lastMessage() }}</p>` })
export class SubscriberComponent {
  constructor(public bus: NotificationBusService) {}
}
```

---

4. Internal Working

4.1 How `@Input()` is compiled

`@Input()` (and `@Output()`) are compile-time metadata, not runtime magic performed by the decorator function itself. The Angular compiler (`ngc` / `@angular/compiler-cli`, via the Ivy pipeline) statically scans the class for `@Input()`-decorated members during compilation and bakes the mapping into the component's generated definition object, `ɵcmp` (`ComponentDef`), specifically its `inputs` map: `{ propertyName: [bindingPropertyName, declaredName, transform?] }`.

At the call site, when the template compiler processes `<app-child [name]="value">`, it doesn't call a setter through the decorator — it generates instruction calls like `ɵɵproperty('name', ctx.value)` in the parent's compiled template function. During change detection, Ivy's `refreshView` walks the binding instructions and, for each input binding, checks `ComponentDef.inputs` to decide: is this a plain field (direct property write, `instance[propName] = value`) or does it have a declared **setter** (in which case it invokes the setter function instead of writing the field directly)? This is why an `@Input()` accessor (`set foo(v)`) fires on every binding update, not just the first — the compiler wires the input write to go through your setter function every time the bound expression's value changes.

Decorators themselves are just `reflect-metadata`-free annotations read by the Angular **compiler** at build time (with Ivy, via static analysis of the AST, not runtime reflection) — by the time the app runs in the browser, there is no `@Input` decorator execution happening at all; it has been fully compiled away into the `inputs` / `outputs` arrays on the component definition.

4.2 How `@Output()` / `EventEmitter` works

Similarly, `outputs` is a static map on `ComponentDef`. A template binding `(itemSelected)="onSelect($event)"` compiles to an instruction that, during template creation (not every CD cycle — only once, at view creation time), calls `.subscribe()` on the child instance's `itemSelected` (an `EventEmitter`, which is an RxJS `Subject` subclass) and wires the subscription's callback to invoke the parent's listener expression with `$event` bound to whatever value was passed to `.emit()`. Angular manages the lifecycle of that subscription tied to the view — when the view is destroyed, this internally-created subscription is torn down automatically. That's why you don't need to manually unsubscribe from `@Output()` bindings in templates — this automatic teardown is Angular's own listener wiring, not a general property of `EventEmitter` / `Subject` itself.

4.3 How signal `input()` differs internally

`input()` does **not** rely on the same "assign to a field / call a setter" plumbing at all. Calling `input()` at class-field-initializer time creates an `InputSignal`, a special signal node registered with the component's `LView` at a specific slot. The Ivy input map for the component now records that this input is **signal-based** (a flag on the input definition, distinguishing "SignalInputDef" from the old direct-assignment style).

When the template writes to a signal input (`ɵɵproperty('name', ctx.value)` still compiles the same way from the parent's perspective), Ivy detects the signal-input flag and instead of writing to a plain field or calling a decorator setter, it calls a special internal `writeToSignalInput()` (or equivalent internal function) that sets the underlying `WritableSignal` backing the `InputSignal` — but only through Angular's internal channel, not through the public API (the signal returned to your class is a read-only-facing `Signal<T>`; the *writable* handle is private to the framework's input-binding machinery). This is why you can call `this.name()` to read it but have no `.set()` / `.update()` available on it from your own code — the child cannot mutate its own input, by construction, unlike the old `@Input()` field which was a completely ordinary mutable class property that nothing stopped you from reassigning locally (a common footgun: reassigning an `@Input()` inside the child looks like it works locally but silently diverges from the parent's value on the next parent-driven change).

Because it's a genuine signal, reading `name()` inside a `computed()` or a template automatically registers a fine-grained reactive dependency — Angular doesn't need to run full component-tree change detection to know this particular consumer needs to re-run; it can (under zoneless/signal-based CD) surgically re-evaluate only what depends on that signal. This is the core architectural reason the Angular team introduced signal inputs: `@Input()` values were "dumb" data landed on the instance by CD's tree walk, whereas `input()` values are reactive nodes in the same dependency graph as every other signal, unlocking zoneless change detection and fine-grained reactivity.

4.4 How `@ViewChild()` / `viewChild()` resolves

View and content queries are also compiled into static instructions (`ɵɵviewQuery`, `ɵɵcontentQuery`) attached to the component definition. During the view's creation and refresh passes, Ivy walks the declared query list against the actual rendered `LView` node tree (or `TNode` tree) looking for matches (by component type, directive type, or template reference name), and populates the result — either directly onto your decorated class field (old API, requires an `ngAfterViewInit` / `ngAfterContentInit` hook to know it's ready) or into an internal signal node that your `viewChild()` call reads from (new API — same underlying resolution timing, just exposed as a signal you can read any time, returning `undefined` until the query actually resolves).

---

5. Edge Cases & Gotchas

1. Input setter timing vs `ngOnChanges`. An `@Input()` setter fires *synchronously as each individual input is written*, in the order Angular processes the `inputs` map — which is not guaranteed to be your declaration order, and is not guaranteed to be *after* all other inputs on the same instance have already been set. If component logic depends on two inputs being consistent with each other, don't rely on doing it inside one input's setter; use `ngOnChanges(changes: SimpleChanges)` instead, which receives **all** the inputs that changed in that CD pass together, or use `ngOnInit` /an `effect()` (signal inputs) if you need all values settled first.

2. `ngOnChanges` fires on reference change only, for objects. If you pass an object into `@Input() config: Config` and the child (or worse, the parent, elsewhere) does `parent.config.someField = 'x'` in place, `ngOnChanges` will **not** fire — Angular's default `ngOnChanges` diffing is a shallow reference/primitive comparison per input, not a deep value comparison. This is one of the most common real-world bugs: "I updated the object but the child didn't re-render." Fix: always replace the object with a new reference (`this.config = { ...this.config, someField: 'x' }`), or use `signal()` / `computed()`-based state where updates naturally happen through immutable `.set()` / `.update()`.

3. Mutating an `@Input()` object mutates the parent's object too. JavaScript objects are passed by reference. If a child mutates fields on an `@Input()`-received object (rather than replacing the whole input), the parent's original object is mutated as a side effect — with no `@Output()`, no explicit "this changed" signal, just silent shared mutable state. This breaks the conceptual one-way-data-flow contract Angular tries to encourage and can cause CD to appear to "miss" updates elsewhere, or double-processing bugs when the same object reference is read by multiple components. Treat all `@Input()` values as read-only from the child's perspective.

4. `EventEmitter` / manual `Subject` subscription memory leaks. If you manually call `.subscribe()` on a service-exposed `Observable` / `Subject` in a component (common in service-based sibling communication) and forget `ngOnDestroy() { this.sub.unsubscribe(); }`, the subscription keeps a reference to the component instance's callback alive for the lifetime of the service (often the whole app, if `providedIn: 'root'`), preventing garbage collection of the destroyed component — a classic Angular memory leak. Mitigations: `takeUntilDestroyed()` (Angular 16+, from `@angular/core/rxjs-interop`), the `async` pipe (auto-unsubscribes when the view is destroyed), or switching the shared state to a `signal()` (no subscription lifecycle to manage at all).

5. `@Output()` bound in a template does NOT need manual unsubscribe. This is the flip side of #4 and a common interview trap: people assume all `EventEmitter` usage requires manual cleanup. It doesn't, *when bound via the template's `(event)` syntax* — Angular's own internal subscription (created when compiling the output binding) is torn down automatically with the view. Manual unsubscribe is only needed when **you** call `.subscribe()` yourself in TypeScript code, not when Angular does it for you via template binding.

6. `static: true` vs `static: false` in `@ViewChild`. `static: true` resolves the query before change detection runs the first time (available in `ngOnInit`), but only works if the queried element is never conditionally rendered (`*ngIf` / `@if`) — if it might not exist yet at that exact moment, Angular can't give you a static reference to it. Anything inside `*ngIf` / `*ngFor` / `@if` / `@for` **must** use `static: false` (the default) and be read no earlier than `ngAfterViewInit`.

7. `@ContentChild` resolves in `ngAfterContentInit`, not `ngAfterViewInit`. Reading a `@ContentChild`-queried value inside `ngOnInit` or even inside `ngAfterViewInit`'s "for view" logic is a frequent source of `undefined`-related bugs — content queries are guaranteed to be resolved specifically by the content hooks (`ngAfterContentInit` / `ngAfterContentChecked`), which run *before* the corresponding view hooks in the same change detection round, but the two query types are conceptually and temporally distinct and mixing them up is an easy mistake.

8. `QueryList` is not a plain array and it's mutable over time. `@ViewChildren` / `@ContentChildren` return a `QueryList<T>`, which updates itself as matching elements are added/removed (e.g., items appearing/disappearing via `*ngFor` / `@for`). You must subscribe to `queryList.changes` (an `Observable`) if you need to react to elements appearing/disappearing after the initial resolution — reading `.toArray()` once and caching it will go stale. This subscription also needs cleanup in `ngOnDestroy`.

9. Signal inputs cannot be reassigned/mutated by the child. Attempting `this.someSignalInput.set(x)` doesn't compile — `input()` returns a read-only `Signal<T>`, not a `WritableSignal<T>`. This is by design (see Internal Working §4.3), and is actually a fix for gotcha #3 above for the signal-input case; but it means any pattern relying on the child locally overriding its input (e.g., "start from the input value, then let the user edit locally") now needs an explicit local signal seeded from the input via `effect()`, rather than just reassigning the input field like you could with `@Input()`.

10. `model()` inputs are exempt from the "read-only" rule, on purpose. Unlike `input()`, `model()` returns a writable signal, specifically because it represents a two-way-bound value the child is expected to update (and have that update flow back to the parent). Don't confuse `input()` (one-way, read-only) with `model()` (two-way, writable) in an interview answer — this distinction is frequently tested.

11. Passing a callback function as an `@Input()` instead of using `@Output()`. Some teams instead pass a plain function into a child as data (`@Input() onSelect!: (item: Item) => void;`) and call it directly. This *technically* achieves child-to-parent communication without `EventEmitter`, but it breaks Angular's binding/CD conventions (no `$event`-style template syntax, harder to test, no automatic teardown semantics), and most style guides/interviewers will flag it as an anti-pattern versus using `@Output()` / `output()` properly.

12. Multiple sibling subscribers racing on a plain `Subject`. With a plain `Subject` (not `BehaviorSubject`), a sibling that subscribes *after* an event was emitted will simply never see it — there's no replay. A common bug: a sibling component that renders after another component fires its "load complete" event misses it entirely because it subscribed too late. If "late subscribers should see the latest state" is a requirement, use `BehaviorSubject`, `ReplaySubject`, or a `signal()` (which always reflects current value to any reader, at any time).

---

6. Interview Questions & Answers

Q1. What is the fundamental data-flow direction of `@Input()` and `@Output()`, and why can't a child use `@Input()` to send data back up to the parent?

`@Input()` is strictly parent → child; `@Output()` is strictly child → parent. Angular's change detection walks the component tree top-down, and input bindings are refreshed as part of that top-down walk — there's no mechanism for a child's local reassignment of an `@Input()`-bound field to propagate back up, because the parent's own bound expression is what re-supplies the value on the next CD pass (it will simply overwrite whatever the child did locally). To send data upward you need an explicit channel the parent listens to — that's exactly what `@Output()` provides.

Interviewer intent: checks that the candidate understands this isn't an arbitrary rule but a consequence of how unidirectional data flow and top-down CD are implemented, not just "Angular says so."

Q2. How would you implement communication between two sibling components that have no parent-child relationship visible to each other?

Via a shared injectable service, provided at a common ancestor (or root, if app-wide) so both siblings receive the same instance through DI. The service exposes state through a `Subject` / `BehaviorSubject` / `ReplaySubject` (or a `signal()` in modern Angular). One sibling calls a method on the service to push a change; the other subscribes to (or reads the signal from) the service to receive it. Neither sibling has a reference to the other — both only depend on the shared service's public API, which keeps them properly decoupled and independently testable.

Q3. What's the difference between `@ViewChild` and `@ContentChild`?

`@ViewChild` queries elements/components that are part of **this component's own template** (its view). `@ContentChild` queries elements/components that were **projected in by the consumer** via `<ng-content>` — i.e., authored in the parent's template, not this component's own. Correspondingly, `@ViewChild` resolves in the view lifecycle hooks (`ngAfterViewInit`), while `@ContentChild` resolves in the content lifecycle hooks (`ngAfterContentInit`), which run earlier in the same CD cycle. Mixing these up (e.g., reading a `@ContentChild` result inside `ngAfterViewInit` and expecting it wasn't already available, or vice versa assuming a `@ViewChild` is ready in `ngAfterContentInit`) is a common source of bugs.

Q4. Why doesn't `ngOnChanges` fire when you mutate a nested property of an object passed via `@Input()`?

`ngOnChanges` diffing (via Angular's default `KeyValueDiffer`-free simple change tracking for `SimpleChanges`) is a reference/primitive-value comparison performed once per input, per change detection pass — it compares the previous bound value to the current bound value using `!==` semantics, essentially. It does **not** perform a deep equality check on object contents. If you mutate a field on the same object instance (`this.config.x = 1`), the reference passed into the input never changes, so Angular sees no difference and does not report a change via `ngOnChanges` (though template bindings reading `config.x` directly, e.g., `{{ config.x }}`, *will* still re-render correctly on the next CD pass because interpolation just re-evaluates the expression regardless — it's specifically the `ngOnChanges` hook and things relying on `SimpleChanges` detection, like `OnPush` components without explicit markForCheck, that miss it). The fix is to always replace the object with a new reference on update (immutability), which is also exactly what makes `OnPush` change detection strategy work correctly.

Interviewer intent: this tests whether the candidate actually understands *why* immutability matters in Angular rather than reciting it as dogma — connects directly to `OnPush` strategy correctness.

Q5. When would you choose a template reference variable over `@ViewChild()` for parent-child communication?

Template reference variables (`#ref`) are the right choice when the interaction is entirely within the same template — e.g., a button in the parent's own HTML calling a method on a sibling element/component declared in that same HTML (`<button (click)="child.reset()">`), with no need to touch the value from the parent's TypeScript class at all. `@ViewChild()` is needed when the parent's **component class** (not just its template) needs programmatic access — e.g., inside a lifecycle hook, inside a method triggered by something other than a template event, or when you need to store the reference for later use across multiple methods. Both can coexist referencing the same element (`#ref` for template use, `@ViewChild('ref')` for class-side use).

Q6. What replaces `EventEmitter` in the modern signal-based Angular API, and what concrete problem does it solve?

`output()` (returning `OutputEmitterRef<T>`) replaces `@Output() x = new EventEmitter<T>()`. It solves the conceptual mismatch of `EventEmitter` extending RxJS `Subject` — which made it look like a general-purpose Observable stream (inviting `.pipe()`, manual `.subscribe()`, and the unsubscribe-management burden that comes with that) when its only intended use is as the backing type for template `(event)` bindings. `output()` is a narrower, purpose-built type: you can only `.emit()` on it, it isn't misused as a general reactive stream, and it integrates cleanly with Angular's push toward signals and zoneless change detection, while still supporting conversion to an Observable via `outputToObservable()` when genuinely needed.

Q7. Explain how `[(value)]="x"` two-way binding syntax is implemented under the hood using plain `@Input` / `@Output`.

It's syntactic sugar Angular's template compiler desugars at compile time: `[(value)]="x"` becomes `[value]="x" (valueChange)="x = $event"`, provided the target component declares both `@Input() value` and `@Output() valueChange` (the `Change`-suffix naming convention is what makes the compiler recognize the pair as bananable). There's no special runtime "two-way" mechanism distinct from ordinary property binding plus event binding — it's purely a naming-convention-driven compiler transform.

Q8. What is the actual difference, internally, between a signal-based `input()` and a decorator-based `@Input()` at the point where the parent's bound value gets written into the child?

With `@Input()`, the Ivy `ComponentDef.inputs` map marks the property either as a plain field (Angular does `instance[propName] = value` directly on the class instance during the tree refresh) or as having a declared accessor (Angular calls your `set propName(value)` setter instead). Either way, the value lands as an ordinary mutable JS class property/field write. With `input()`, the corresponding entry in the input map is flagged as signal-based, and instead of a plain field write, Ivy writes the incoming value into the internal `WritableSignal` node backing the `InputSignal` through a framework-private channel — the public signal object your class holds (`this.name`) only exposes the read-facing `Signal<T>` API (callable to read, no `.set()`), so the child cannot mutate it directly the way it could reassign an ordinary `@Input()` field. This also means reading a signal input inside a `computed()`/effect/template automatically registers a fine-grained reactive dependency in Angular's signal graph, whereas a plain `@Input()` field read has no such dependency-tracking properties on its own.

Interviewer intent: distinguishes candidates who've only used signal inputs superficially from those who understand *why* the framework introduced them — the read-only-by-construction guarantee and the fine-grained reactive graph integration, not just "it's the new syntax."

Q9. You have a grandparent component, a parent, and a grandchild, and the grandchild needs a piece of data that only the grandparent has. What are your options, and which is generally preferred?

Option A: "input drilling" — pass the value as an `@Input()`/`input()` from grandparent to parent, and parent re-declares its own `@Input()`/`input()` and passes it straight through to the grandchild, purely as a pass-through with no use at the parent level. This works but doesn't scale: every intermediate level has to know about and forward data it doesn't actually need, and adding a new intermediate level later means touching every layer in between. Option B (generally preferred for anything beyond one extra hop): put the value in a shared service (state service, or in more complex apps a store), inject it directly in the grandchild, and skip the intermediate parent entirely. This avoids drilling and keeps intermediate components decoupled from data they don't use. Option A remains reasonable for very shallow, stable hierarchies where the intermediate

component conceptually owns/passes that data anyway (e.g., a well-defined "props down" component library); Option B is preferred as the hierarchy deepens or the data becomes cross-cutting.

Q10. Why is it safe to skip manual unsubscription for an `@Output()` bound via `(event)="handler($event)"` in a template, but not safe to skip it when you manually call `.subscribe()` on that same `@Output()` `EventEmitter` from TypeScript code?

When Angular compiles a template's `(event)` binding, it generates its own internal subscription to the emitter as part of view creation, and that subscription's teardown is wired into the view's own destruction lifecycle — Angular unsubscribes it for you automatically when the view is destroyed. This is a property of *how the template compiler wires listener bindings*, not an inherent property of `EventEmitter`. If you instead take that same `@Output() foo = new EventEmitter<T>()` and manually call `someChildInstance.foo.subscribe(...)` yourself from a service or a parent's TypeScript code (bypassing the template binding), you've created an ordinary RxJS subscription with no framework-managed teardown, and you are fully responsible for calling `.unsubscribe()` (typically in `ngOnDestroy`) — Angular has no idea you created that manual subscription and cannot clean it up for you.

Interviewer intent: many candidates parrot "you don't need to unsubscribe from Outputs" as a blanket rule without understanding it's specifically about the template-binding-generated subscription, not a magic property of the `EventEmitter` type itself — this question surfaces that gap.

Q11. How does `@Input({ required: true })` differ in enforcement from `input.required()`?

`@Input({ required: true })` (Angular 16+) is enforced by the Angular **template type-checker** at compile time when using the Ivy compiler in full/strict template type-checking mode — if a parent template omits a binding for a required decorator input, the build fails. `input.required<T>()` provides the same static/compile-time enforcement via the compiler, but additionally the signal itself throws a runtime error if read before Angular has actually written a value into it (which shouldn't normally happen given the compiler already prevents unbound usage, but provides a stronger runtime guarantee/type — `input.required()`'s return type is `Signal<T>`, non-nullable, with no `| undefined` in its type signature, whereas a merely-decorated `@Input({ required: true })` field is still just a plain class property typed as `T` that you the developer promised (often via `!`) is always assigned — the compiler checks call sites, not runtime).

Q12. Design a solution: Component A (deeply nested) needs to trigger a refresh in Component B (a distant, unrelated part of the tree), and B needs to tell A when the refresh completes. Walk through the architecture.

Introduce a shared `RefreshService` (`providedIn: 'root'`, or scoped to their nearest common ancestor if the interaction should be limited to a subtree). Expose two channels: a "request" channel A pushes to (`Subject<void>` or a plain method the service exposes, e.g. `requestRefresh()`, backed by a `Subject`/signal `refreshRequested`), and a "completed" channel B pushes to (`refreshCompleted$` or a signal `lastCompletedAt`). Component A injects the service and calls `refreshService.requestRefresh()`; Component B injects the same service, subscribes to (or reads the signal for) the request channel, performs its refresh work, then calls `refreshService.notifyCompleted()`. Component A subscribes to (or reads the signal for) the completion channel to react (e.g., re-enable a "Refresh" button, show a toast). If using RxJS, both subscriptions need explicit teardown (`takeUntilDestroyed()` is the cleanest modern approach); if using signals for both channels, no manual subscription management is needed at all — this is a strong argument

for using `signal()`-based service state for this kind of one-to-one-but-topologically-distant communication in newer codebases.

Interviewer intent: this is a synthesis question — checks whether the candidate can compose the primitives (service, Subject/signal, injection scope, cleanup) into a coherent bidirectional design rather than just reciting individual pattern definitions in isolation.

---

7. Quick Revision Cheat Sheet

- **Parent → Child data**: `@Input()` (decorator) or `input()` / `input.required()` (signal). Signal inputs are read-only signals; decorator inputs are plain mutable fields (or setters).
- **Child → Parent event**: `@Output() x = new EventEmitter<T>()` (decorator) or `output<T>()` (signal function, not itself a signal — an `OutputEmitterRef`).
- **Two-way binding**: `[(x)]` desugars to `[x] (xChange)`. Modern replacement: `model()` — a writable signal that auto-emits on `.set()` / `.update()`.
- **Parent commanding child directly**: `@ViewChild()` / `@ViewChildren()` (decorator, resolves in `ngAfterViewInit`) or `viewChild()` / `viewChildren()` (signal, same timing, `undefined` until resolved).
- **Reading projected content**: `@ContentChild()` / `@ContentChildren()` (resolves in `ngAfterContentInit` — earlier than view hooks) or `contentChild()` / `contentChildren()` (signal equivalents).
- **Same-template, no class code**: template reference variable `#ref`, used directly in bindings like `(click)="ref.method()"`.
- **Siblings / unrelated / cross-cutting**: shared injectable service with `Subject` (no replay), `BehaviorSubject` (has current value, replays last to late subscribers), `ReplaySubject` (replays N), or `signal()` (no subscription management, always current, integrates with `computed` / `effect`).
- **Grandparent ↔ grandchild**: avoid input/output drilling through every level; use a shared service instead once more than one hop is involved.
- **Compiler internals**: `@Input` / `@Output` become static entries in Ivy's `ComponentDef.inputs` / `outputs`; there's no runtime decorator execution left after compilation. Template `(event)` bindings auto-subscribe/auto-teardown; manual `.subscribe()` calls do not.
- **Signal inputs vs decorator inputs**: signal inputs write through a framework-private writable channel into a genuine signal node (fine-grained reactive, read-only from the child); decorator inputs are ordinary field writes or setter invocations (mutable, no built-in reactive graph participation — that's what `ngOnChanges` exists to patch over).
- **Gotchas to always mention**: object-typed inputs need a new reference to trigger `ngOnChanges` / `OnPush`; mutating an `@Input()` object mutates the parent's object too; `static: true` ViewChild only works for always-present elements; `QueryList` needs `.changes` subscription for live updates; manual `Subject` subscriptions from services need explicit `ngOnDestroy` cleanup (or `takeUntilDestroyed()`), template-bound `@Output()`s don't.

Created By - Durgesh Singh